

SOLUTION BRIEF · TECHNICAL EVALUATION

Shared memory for every AI coding agent.

Claude Code, Minimax, and Cursor working a real Pulumi + AWS Landing Zone estate from one queryable memory on TiDB Cloud - isolated per team, spanning repos, answering what a vector store can't.

Audience platform engineering

Store mem9 → TiDB Cloud

Access Model Context Protocol

Scope team acme · repos pulumi + lza

What you asked, answered.

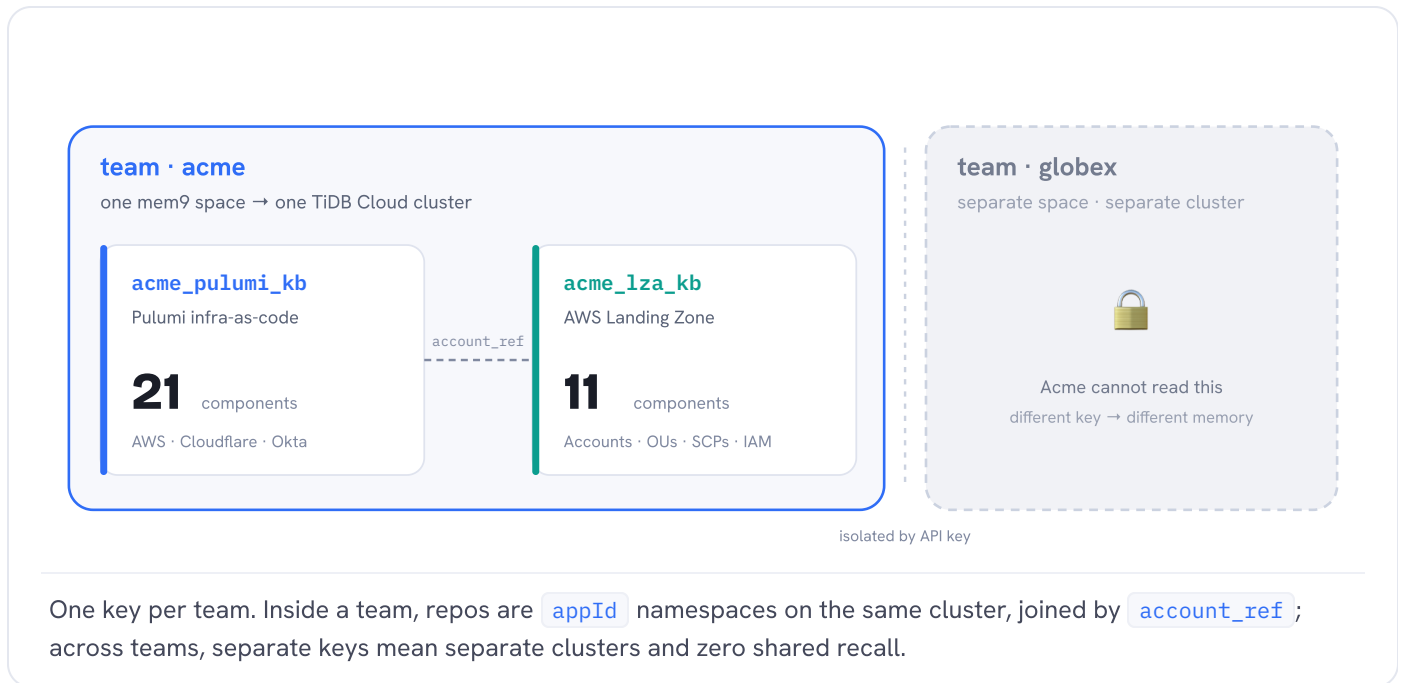
Bottom line. mem9 gives a team of AI coding agents one shared, queryable memory on TiDB Cloud. Agents read before they build and write back when done - so none starts cold or duplicates work. It is isolated per team, an agent can work across repos in a single session, and it answers the one question a vector store cannot: the transitive blast radius of a change. Every screenshot in this brief is a live read or write against TiDB Cloud.

YOUR QUESTION ON THE CALL	THE ANSWER	SHOWN IN
Can teams keep isolated knowledge bases - no team reading another's?	Yes. One TiDB cluster per team (a mem9 space). A team's key reaches only its own memory; isolation is enforced by the platform, not app code.	§ The model
One MCP across multiple repos / clusters?	Yes. Within a team, repos are <code>appId</code> namespaces on one cluster and the agent routes to the right one. Across teams, separate keys = separate clusters.	§ The model
Can one task span both repos - new AWS account in LZA <i>and</i> an S3 bucket for it in Pulumi?	Yes. A single agent session writes the account to the LZA namespace and the bucket to the Pulumi namespace, joined by <code>account_ref</code> - one MCP, two repos.	§ One task, two repos
Self-hosted option?	Yes, with one caveat - full-text search is TiDB Cloud only today; vector + graph run self-hosted.	§ Deployment

Demonstrated live with three different coding agents - Claude Code, Minimax, and Cursor - against a single TiDB Cloud cluster. The tools are interchangeable; the shared memory is the point.

Team = cluster. Repo = namespace.

mem9 maps onto how a team already works. A **team** is a mem9 **space** - one key, one isolated TiDB Cloud cluster. A **repo** is an `appId` namespace inside it, so repos share a cluster but keep separate recall scopes. Another team is another cluster, reachable only with its own key.



Each team's space sits on its own managed TiDB Cloud cluster. Agents never handle raw database credentials - the mem9 API key is the only secret.

One task, two repos.

You asked whether one session could create a new AWS account in the LZA repo *and* the S3 bucket for it in the Pulumi repo. It can. A single agent, given one plain-language request, writes the account to `acme_lza_kb` and the bucket to `acme_pulumi_kb` - two separate namespaces - and links them with `account_ref`. No second session, no manual hand-off.

Left: one activity feed shows the same agent writing to **both** repos. Right: the `account_ref` join - the new **data-platform** account and its Pulumi exports bucket, created together and linked.

Why it matters. The agent doesn't need to be told which repo owns what. It routes each write to the right namespace and records the cross-repo link, so the next session - in either repo - already knows the account and its bucket belong together.

Three tools, one memory, warm hand-offs.

A second scenario, the one in the video: production is complete, staging has drifted by four components. Three different agents bring it back to parity - each reading the shared memory before it builds and writing back when done. The developer only states the goal; the standing instruction does the rest.

1 • Claude Code

```
dev types "Bring
staging up to parity
with production -
start with the
static-assets bucket
and its CDN."
```

Queries the KB, finds the gap, composes the libraries, writes both back. **Gap 4 → 2.**

2 • Minimax

```
dev types "Keep
bringing staging to
parity with prod - it
still needs the
admin-portal DNS."
```

Opens cold. Reads the shared session log, sees what was just built, continues with admin-DNS. **Gap 2 → 1.**

3 • Cursor

```
dev types "Finish
bringing staging to
parity - add the
admin SSO app."
```

Traces what prod's admin-SSO depends on, confirms it redirects to a DnsRecord, builds the SSO app. **Gap 1 → 0.**

The gap closing - real writes to mem9:

Three tools, one memory, live.

Every query and write from Claude Code, Minimax, and Cursor lands in the same mem9 space. Watch the staging gap close as each tool reads the shared log and writes back.

ACTIVITY

No activity yet. Launch a CLI and ask it to query the knowledge base.

KNOWLEDGE GRAPH

All Lib Prod Staging Org



STAGING PARITY

4 missing in staging

✓ analytics-db	RDS	in staging
✓ data-exports	S3	in staging
✗ static-assets	S3	missing
✗ assets-dns	Cloudflare	missing

START Feed empty; staging-parity strip shows 4 missing.

mem9

Infra Knowledge Base

acme · pulumi + tza

TiDB Cloud

Reset

Live

Cross-repo

Dependencies

Search

Config

Three tools, one memory, live.

Every query and write from Claude Code, Minimax, and Cursor lands in the same mem9 space. Watch the staging gap close as each tool reads the shared log and writes back.

ACTIVITY

minimax

WRITE

Created acme-staging-admin-dns, a Cloudflare DNS matching the production admin pattern.

minimax

QUERY

The user is continuing staging work.

claude-code

WRITE

Created acme-staging-assets-dns, a Cloudflare DNS fronting acme-staging-static-assets.

claude-code

WRITE

The acme-staging-static-assets resource is an S3 bucket composing the S3Bucket library.

KNOWLEDGE GRAPH

All Lib Prod Staging Org

Library S3 RDS Cloudflare Okta Account

● pulumi

● tza

STAGING PARITY

1 missing in staging

✓ analytics-db	RDS	in staging
✓ data-exports	S3	in staging
✓ static-assets	S3	in staging
✓ assets-dns	Cloudflare	in staging

WARM HAND-OFF

Minimax opens cold, reads the shared log first ("continuing staging work"), then builds - no re-briefing. Gap 1.

Three tools, one memory, live.

Every query and write from Claude Code, Minimax, and Cursor lands in the same mem9 space. Watch the staging gap close as each tool reads the shared log and writes back.

ACTIVITY

cursor
WRITE
Created acme-staging-admin-sso, an Okta SSO app whose redirect URI points at acme-staging-admin-dns.

cursor
WRITE
Staging is now at parity with production.

minimax
WRITE
Created acme-staging-admin-dns, a Cloudflare DNS matching the production admin pattern.

minimax
QUERY
The user is continuing staging work.

claude-code
WRITE
Created acme-staging-assets-dns, a Cloudflare DNS fronting acme-staging-static-assets.

claude-code
WRITE
The acme-staging-static-assets resource is an S3 bucket composing the S3Bucket library.

KNOWLEDGE GRAPH

All Lib Prod Staging Org



STAGING PARITY

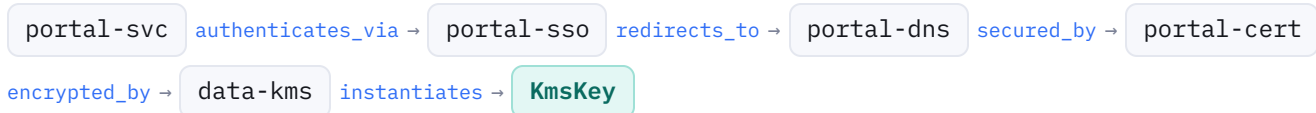
at parity ✓

✓ analytics-db	RDS	in staging
✓ data-exports	S3	in staging
✓ static-assets	S3	in staging
✓ assets-dns	Cloudflare	in staging

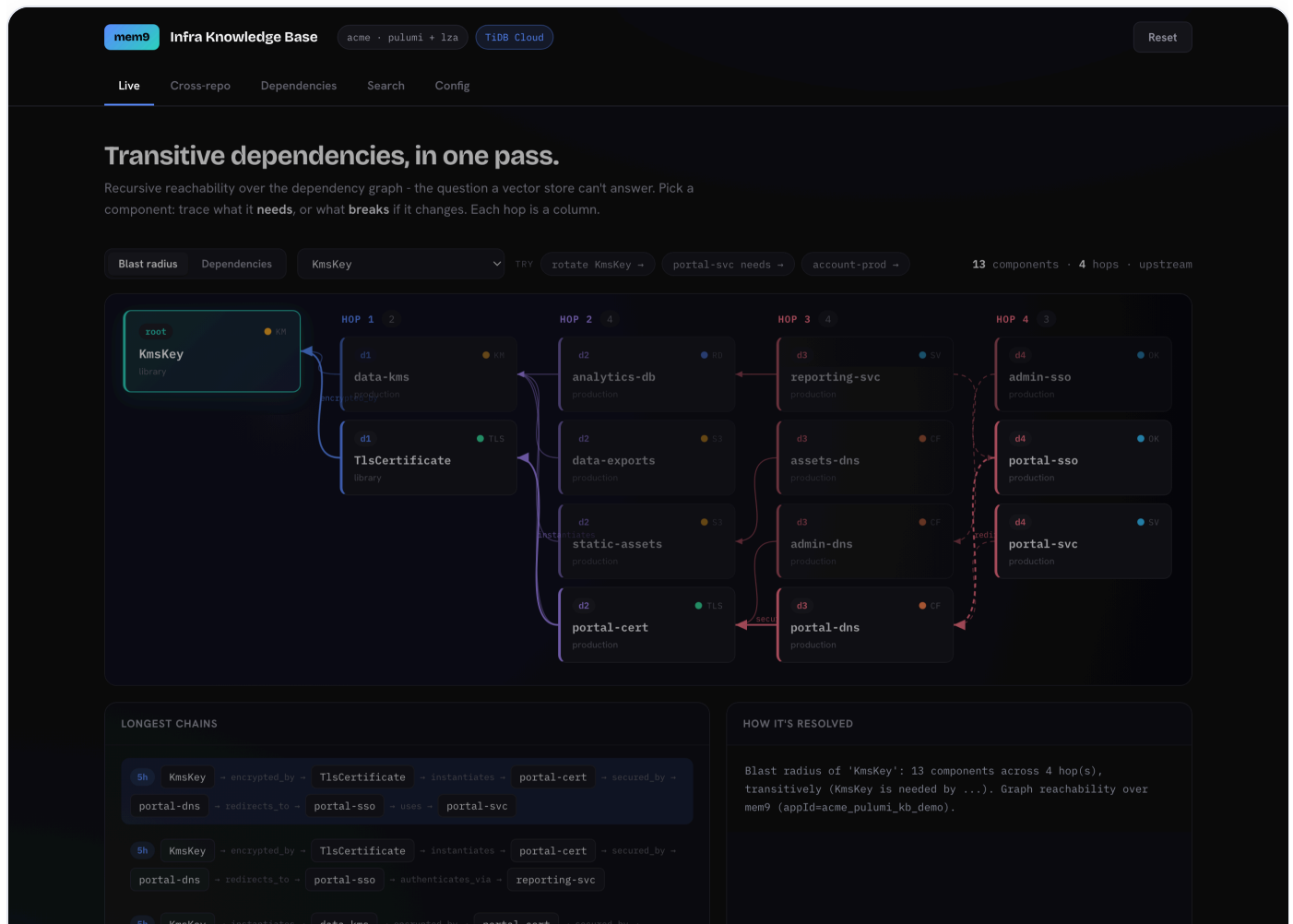
PARITY One timeline across all three tools; the last write reads "Staging is now at parity with production." Gap 0.

The question similarity can't answer.

A vector store finds things that *look* related. It cannot compute a transitive closure - "what, exactly, breaks if I change this?" That is graph reachability. mem9 records every `depends_on` and composition edge; one recursive traversal walks the chain to any depth.



Run that in reverse from `KmsKey` - "what breaks if we rotate it?" - and the answer is **13 components across four depths**. No flat search surfaces this; only walking the graph does.



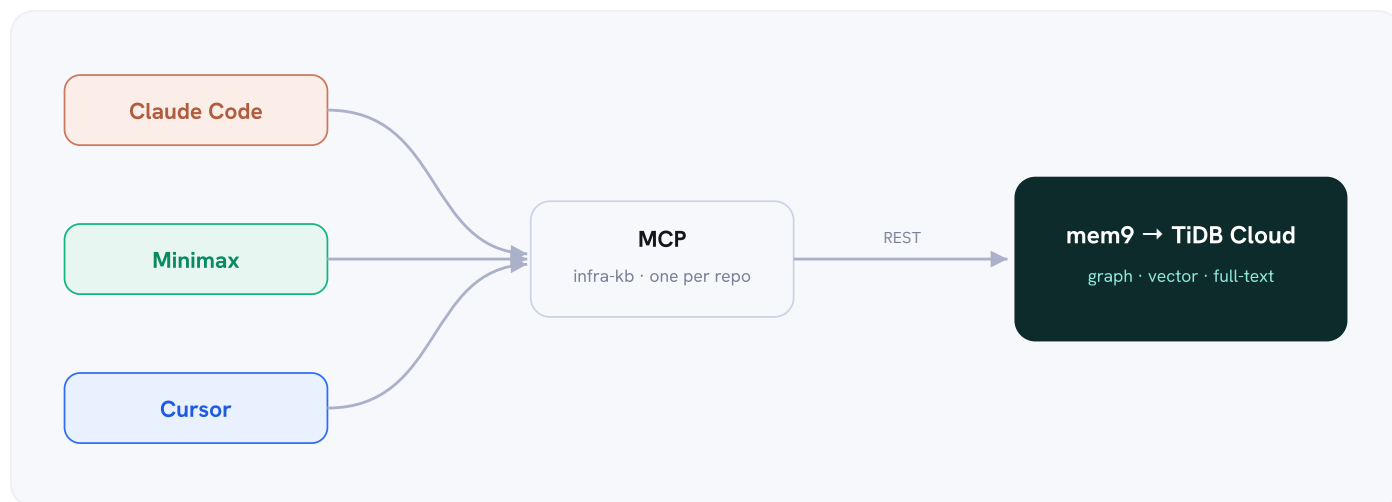
One engine, not four

APPROACH	GRAPH REACHABILITY	SEMANTIC RECALL	ATOMIC AGENT STATE
S3 / filesystem	no traversal	no	no transactions
Vector DB only	similarity ≠ reachability	yes	no
Four separate systems	partial, with ETL	yes	no cross-system atomicity
mem9 → TiDB Cloud	recursive traversal	vector + full-text	one transaction

Graph, vector embeddings, and full-text live in the same store - the same recall that answers "*what encrypts production data*" also computes the blast radius above. No separate vector DB, no ETL, consistent across concurrent writes from all three tools.

S3 is the filing cabinet for artifacts. mem9 is the brain agents query.

One store. One plug. One key.



MCP is the plug

Every tool loads the same Model Context Protocol config - one named server per repo. No custom glue per tool.

One store, every pattern

Relational state, the dependency graph, vector embeddings, and full-text all live in the same memory.

Just an API key

mem9 manages the TiDB Cloud cluster. Agents read and write through one key and never touch credentials.

The standing instruction

The developer types a normal request. The prescriptive part lives in each tool's rules file (`CLAUDE.md` / `AGENTS.md` / `.cursorrules`), so the agent queries first, composes, and writes back on every run with no per-task hand-holding.

mem9

Infra Knowledge Base

acme · pulumi + lza

TiDB Cloud

Reset

Live

Cross-repo

Dependencies

Search

Config

Behavior is configuration, not code.

The developer just types a normal request - "bring staging to parity." This standing instruction, loaded into every tool's rules file, makes the agent query first, compose, and write back on its own. No per-task hand-holding.

STANDING INSTRUCTION · CLAUDE.MD / AGENTS.MD / .CURSORRULES

Before you write or change any infrastructure - automatically, every task:

1. Query first. query_knowledge_base on the infra-kb MCP for the repo you're touching: what exists, how it connects, recent log.
2. Compose. Build from libs/ (S3Bucket, PostgresDatabase, DnsRecord, SsoApplication, KmsKey, ...). Never raw aws.*/cloudflare.*/okta.*.
3. Conventions. acme-<env>-<name> naming; required tags; DnsRecord proxied; SSO redirect -> DnsRecord; set account_ref (prod/sandbox).
4. Blast radius. Before changing a shared component, trace what depends on it.
5. Write back. After any change, call write_component to record it - or the next tool, and the next session, start cold.

The human never says these steps. They say what they want; you do this on your own.

MCP SERVERS · ONE ENTRY PER REPO

```
{
  "mcpServers": {
    "infra-kb-pulumi": { "command": "python", "args": ["-m", "src.mcp_server"] },
    "infra-kb-lza": { "command": "python", "args": ["-m", "src.mcp_server"] }
  }
}
```

query_knowledge_base · write_component -> hybrid recall + atomic writes

DASHBOARD · CONFIG

The five-step standing instruction every tool loads, and the one-entry-per-repo MCP wiring that makes it automatic.

Deployment

- **TiDB Cloud (shown here):** graph + vector + full-text, fully managed; nothing to host.
- **Self-hosted:** available - vector and graph run on-prem. Full-text search is TiDB Cloud only today, so this brief uses Cloud to show hybrid recall end to end.

mem9 · TiDB Cloud - shared memory for AI coding agents · v3. Live demo: Claude Code · Minimax · Cursor over team acme (repos pulumi + lza). Every figure is a live read/write against TiDB Cloud.